



ASTRAQUANTUM TECH
SUMMER OF CYBERSECURITY 2026

WEEK 7 PRACTICAL ASSIGNMENT

Cloud Security - Hands-on Learning & Security Assessment
IAM • Storage Security • Cloud Risks • Identity Management

Student Name	Muhammad Raza
Date	September 19, 2026
Instructor	AstraQuantum Tech Team
Program	AstraQuantum Tech - Summer of Cybersecurity 2026

Table of Contents

1. Executive Summary	3
2. Scope	3
3. Methodology	3
4. Lab Summaries	4
Lab 1 - Intro to AWS IAM Enumeration (Pwned Labs)	4
Lab 2 - flaws.cloud, Level 1	5
Lab 3 - The Big IAM Challenge, Challenge 1 (TryHackMe substitute)	6
5. Findings	7
6. Risk	9
7. Impact	10
8. Recommendation	10
9. Evidence	10
10. Cloud Security Questions	10
11. Conclusion	11

1. Executive Summary

This report documents the completion of the Week 7 Cloud Security practical assignment, focused on identity and access management (IAM) and cloud storage security in AWS. Task A was satisfied with two free, hands-on labs: "Intro to AWS IAM Enumeration" on Pwned Labs (an IAM-focused activity) and flaws.cloud Level 1 (a storage/cloud-risk activity). Because the assigned TryHackMe cloud-security rooms were marked Premium on my account, Task B was satisfied using the free alternative permitted by the assignment brief - The Big IAM Challenge, Challenge 1 - which also touches IAM and S3.

Across the three labs, three cloud-security findings were identified and documented: an overly permissive IAM policy using action and resource wildcards, a publicly listable S3 bucket exposing a hidden file, and a public S3 bucket exposed through a wildcard Principal in its bucket policy. All three findings point to the same underlying theme - permissions and access controls that are broader than necessary, either on the identity side (IAM policies) or the resource side (S3 bucket policies) - and each is mapped to a concrete, practical remediation.

2. Scope

Testing was strictly limited to the approved lab platforms and intentionally vulnerable environments listed in the assignment brief. No real organization's cloud account, website, bucket, API, or IP address was tested, and no personal production AWS/Azure/GCP resources were used for attack practice. Only the temporary/dedicated credentials supplied by each lab platform were used.

- Pwned Labs - Intro to AWS IAM Enumeration (<https://app.pwnedlabs.io/labs/intro-to-aws-iam-enumeration>)
- flaws.cloud - Level 1 (<http://flaws.cloud/>)
- The Big IAM Challenge - Challenge 1 (<https://bigiamchallenge.com/challenge/1>), used as the free substitute for Task B since the assigned TryHackMe cloud rooms were Premium-locked

3. Methodology

Each lab was approached with the same lightweight, junior-cloud-analyst methodology recommended in the assignment brief:

- Configure access - set up the AWS CLI or browser session with the credentials/access provided by the lab.
- Enumerate - identify the current identity, its attached/inline policies, and any accessible storage resources.
- Identify the weakness - pinpoint the specific misconfiguration (wildcard permissions, public bucket listing, permissive bucket policy).
- Validate - confirm the weakness is exploitable by retrieving the flag or hidden evidence.
- Assess & document - record the affected control, likely attacker gain, risk level, and a practical recommendation.

4. Lab Summaries

Lab 1 - Intro to AWS IAM Enumeration (Pwned Labs)

Lab / Challenge name and URL: Intro to AWS IAM Enumeration - <https://app.pwnedlabs.io/labs/intro-to-aws-iam-enumeration>

Main objective: To configure the AWS CLI with provided credentials and systematically map the IAM environment to uncover user permissions, inline policies, and attached managed policies.

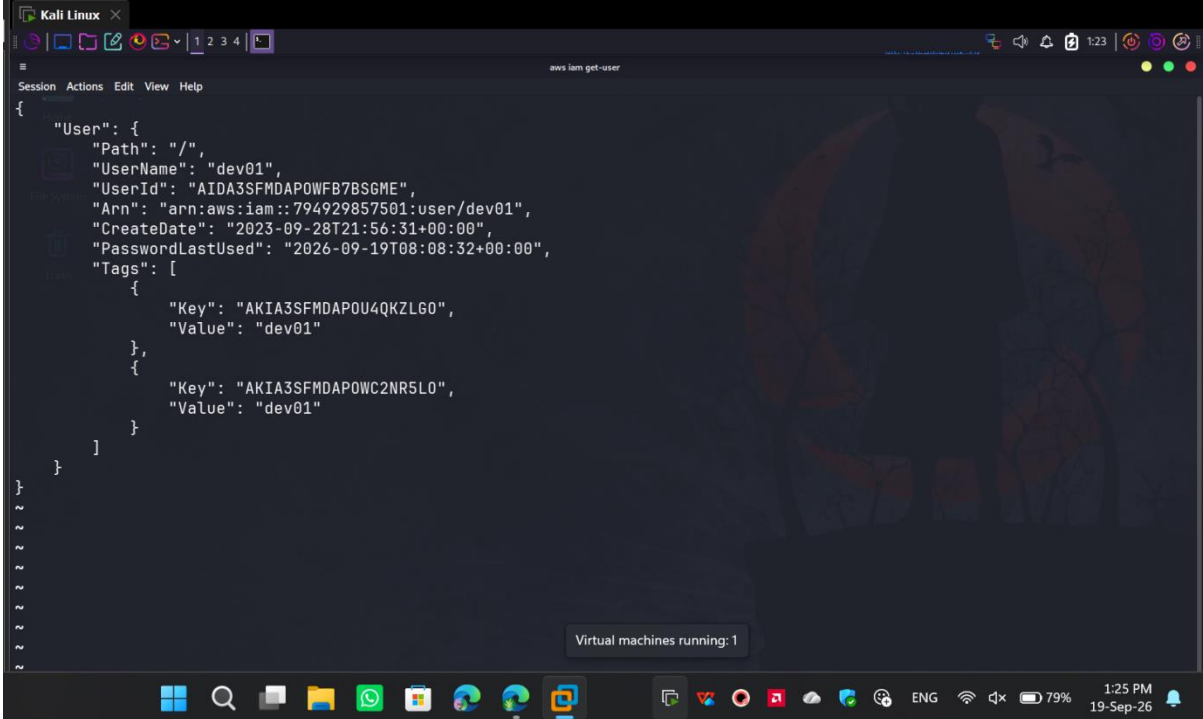
Cloud service involved: AWS Identity and Access Management (IAM)

Security issue observed: Overly permissive IAM policies utilizing wildcards (*) for critical service actions (e.g., GuardDuty and AWS Organizations) applied to all resources.

What caused the weakness?: The policy was authored prioritizing convenience over security, granting broad action arrays rather than explicitly defining necessary actions and restricting them to specific Resource ARNs.

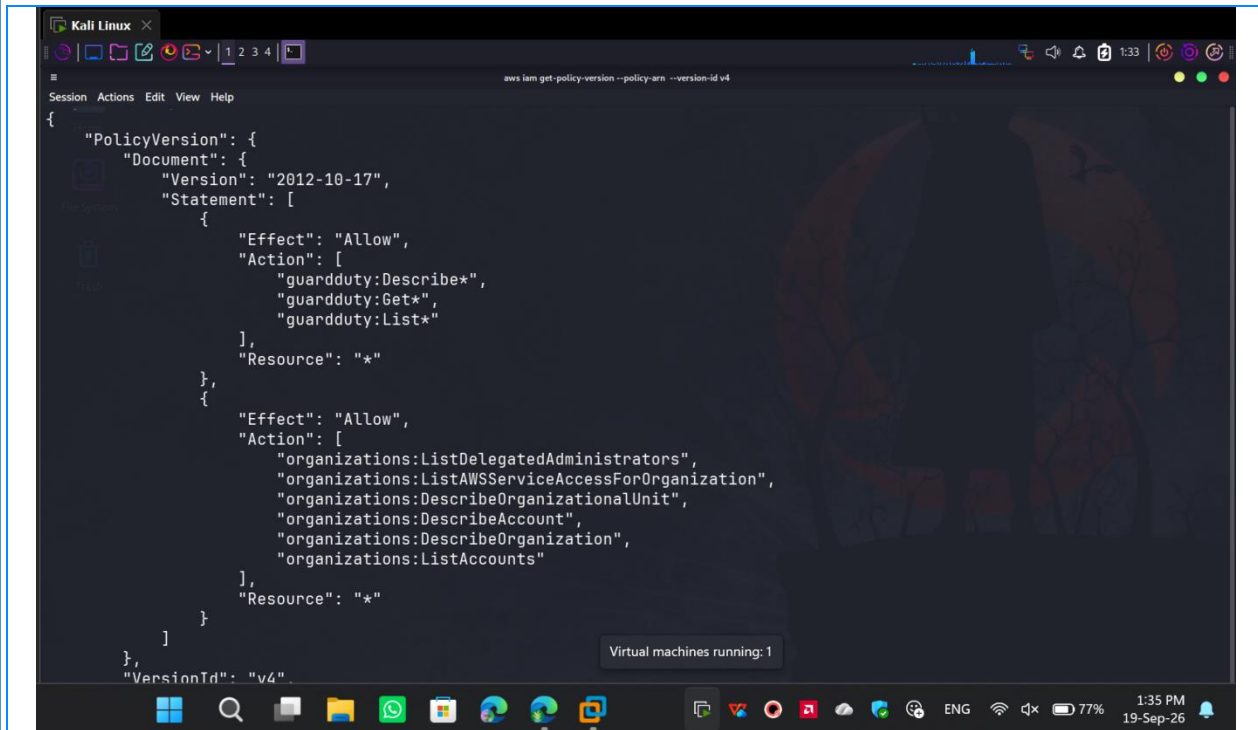
What could an attacker or unauthorized user gain?: An attacker compromising the dev01 user could utilize these excessive permissions to perform unauthorized reconnaissance across the entire AWS Organization or manipulate threat detection settings (GuardDuty).

What security control would reduce the risk?: Strict enforcement of the Principle of Least Privilege by replacing wildcards with specific, necessary API actions limited exclusively to required resource ARNs.



```
aws iam get-user
{
  "User": {
    "Path": "/",
    "UserName": "dev01",
    "UserId": "AIDA3SFMDAP0WFB7BSGME",
    "Arn": "arn:aws:iam::794929857501:user/dev01",
    "CreateDate": "2023-09-28T21:56:31+00:00",
    "PasswordLastUsed": "2026-09-19T08:08:32+00:00",
    "Tags": [
      {
        "Key": "AKIA3SFMDAP0U4QKZLG0",
        "Value": "dev01"
      },
      {
        "Key": "AKIA3SFMDAP0WC2NRSLO",
        "Value": "dev01"
      }
    ]
  }
}
```

Figure 1 - Initial access confirmed as user 'dev01' via the AWS CLI (aws iam get-user).



```
{
  "PolicyVersion": {
    "Document": {
      "Version": "2012-10-17",
      "Statement": [
        {
          "Effect": "Allow",
          "Action": [
            "guardduty:Describe*",
            "guardduty:Get*",
            "guardduty:List*"
          ],
          "Resource": "*"
        },
        {
          "Effect": "Allow",
          "Action": [
            "organizations:ListDelegatedAdministrators",
            "organizations:ListAWSServiceAccessForOrganization",
            "organizations:DescribeOrganizationalUnit",
            "organizations:DescribeAccount",
            "organizations:DescribeOrganization",
            "organizations:ListAccounts"
          ],
          "Resource": "*"
        }
      ]
    },
    "VersionId": "v4"
  }
}
```

Figure 2 - Policy version 'v4' exposing overly permissive wildcard actions applied to all resources.

Lab 2 - flaws.cloud, Level 1

Lab / Challenge name and URL: flaws.cloud - Level 1 - <http://flaws.cloud/>

Main objective: To identify the underlying cloud storage mechanism hosting the target website and enumerate its contents to locate hidden files without any prior authentication.

Cloud service involved: Amazon Simple Storage Service (S3)

Security issue observed: An S3 bucket hosting a static website was configured to allow the s3:ListBucket action for any unauthenticated internet user.

What caused the weakness?: When configuring the bucket for static website hosting, the administrator applied a bucket policy or Access Control List (ACL) that overly provisioned read access to the entire bucket rather than just granting s3:GetObject on the specific web assets.

What could an attacker or unauthorized user gain?: By listing the bucket's contents, an attacker can discover hidden administrative pages, backup archives, or sensitive configuration files that were never meant to be publicly linked or indexed.

What security control would reduce the risk?: Removing the public bucket listing permissions. S3 buckets should block public access by default, and static websites should ideally be served through a CDN (like CloudFront) using Origin Access Control (OAC), rather than making the bucket itself publicly readable.

```
aws s3 ls s3://flaws.cloud --no-sign-request
2017-03-13 20:00:38      2575 hint1.html
2017-03-02 20:05:17      1707 hint2.html
2017-03-02 20:05:11      1101 hint3.html
2024-02-21 18:32:41      2861 index.html
2018-07-10 09:47:16     15979 logo.png
2017-02-26 17:59:28         46 robots.txt
2017-02-26 17:59:30      1051 secret-dd02c7c.html
```

Figure 3 - Unauthenticated AWS CLI request successfully listing the contents of the flaws.cloud S3 bucket.

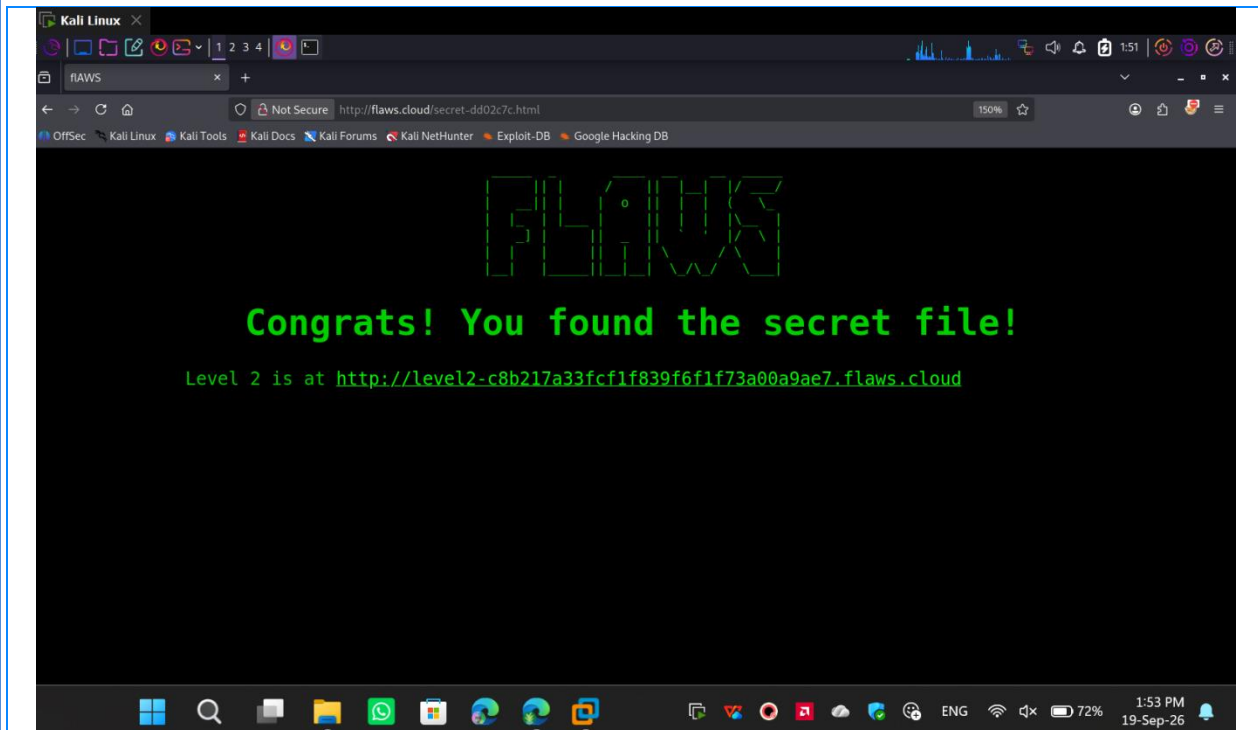


Figure 4 - Accessing the hidden secret file discovered via the exposed bucket directory.

Lab 3 - The Big IAM Challenge, Challenge 1 (TryHackMe substitute)

Lab / Challenge name and URL: The Big IAM Challenge - Challenge 1 -

<https://bigiamchallenge.com/challenge/1> (used in place of the assigned TryHackMe cloud-security room, which was marked Premium on my account)

Main objective: To analyze a resource-based AWS IAM policy, identify public exposure, and extract a hidden flag using the AWS CLI.

Cloud service involved: AWS IAM & Amazon S3

Security issue observed: A resource-based S3 bucket policy utilizing a wildcard (*) in the Principal element, allowing unauthenticated listing of the files/ directory and unauthenticated retrieval of objects.

What caused the weakness?: The administrator granted broad s3:GetObject and s3:ListBucket permissions to * without enforcing a strict trust boundary, such as an aws:SourceIp or a specific AWS Account ID condition.

What could an attacker or unauthorized user gain?: Any unauthenticated internet user can query the bucket, list its contents, and download sensitive files.

What security control would reduce the risk?: Replace the "*" principal with explicitly trusted IAM identities, and enforce AWS "Block Public Access" at the account level.

```
Start the challenge here, you have the aws cli configured. Try executing: aws sts get-caller-identity
> aws sts get-caller-identity
{
  "UserId": "AROAZSFITKRSYE6ELQP2Q:iam_shell",
  "Account": "657483584613",
  "Arn": "arn:aws:sts::657483584613:assumed-role/shell_basic_iam/iam_shell"
}

> aws s3 ls s3://thebigiamchallenge-storage-9979f4b/files/ --no-sign-request
2023-06-05 19:13:53      37 flag1.txt
2023-06-08 19:18:24   81889 logo.png

> aws s3 cp s3://thebigiamchallenge-storage-9979f4b/files/flag1.txt . --no-sign-request
download failed: s3://thebigiamchallenge-storage-9979f4b/files/flag1.txt to ./flag1.txt [Errno 30] Read-only file system:
'/var/task/flag1.txt.EcA10E5D'

Completed 37 Bytes/37 Bytes (882 Bytes/s) with 1 file(s) remaining
> aws s3 cp s3://thebigiamchallenge-storage-9979f4b/files/flag1.txt - --no-sign-request
{wiz:exposed-storage-risky-as-usual}
```

Figure 5 - Exploiting the permissive S3 bucket policy using the AWS CLI (aws s3 cp redirected to stdout), revealing the flag {wiz:exposed-storage-risky-as-usual}.

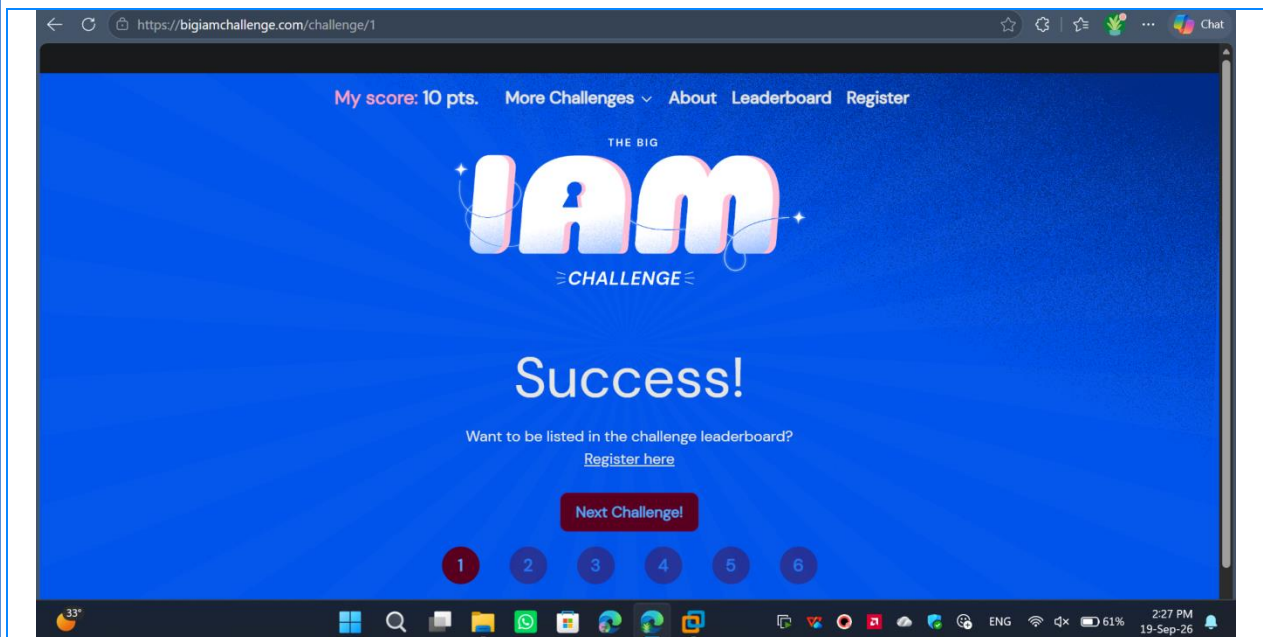


Figure 6 - Challenge 1 completion verified on the platform, with the system clock (19-Sep-26) visible to satisfy academic-integrity requirements.

5. Findings

Finding CLOUD-01: Overly Permissive IAM Policy via Action and Resource Wildcards

Finding ID	CLOUD-01
Title	Overly Permissive IAM Policy via Action and Resource Wildcards
Cloud Service	AWS IAM
Severity	Medium
Description	During the IAM enumeration lab, inspection of the attached policy versions for the dev01 user revealed a highly permissive policy. Version 4 of the policy grants sweeping permissions across AWS Organizations and GuardDuty (e.g., guardduty:*) with the target resource defined as * (all resources).
Why is this risky?	Using wildcards (*) violates the principle of least privilege. It grants the user far more access than practically required for standard development tasks.
Potential Impact	If the dev01 credentials are compromised, an attacker inherits these broad privileges, allowing them to map the entire AWS Organization structure or potentially blind security logging tools across the account.
Recommendation	Audit the dev01 policy and replace the * wildcards with explicit, necessary actions. Restrict the "Resource" block to specific ARNs rather than allowing access to all resources in the account.
Reference / Source	Pwned Labs (Intro to AWS IAM Enumeration); AWS Well-Architected Framework – Security Pillar.

Finding CLOUD-02: Publicly Listable Amazon S3 Bucket Leading to Information Disclosure

Finding ID	CLOUD-02
Title	Publicly Listable Amazon S3 Bucket Leading to Information Disclosure
Cloud Service	AWS S3
Severity	High
Description	During the assessment of the flaws.cloud domain, an unauthenticated AWS CLI query (aws s3 ls s3://flaws.cloud --no-sign-request) successfully returned the full directory listing of the underlying S3 bucket. This directory listing exposed a hidden file (secret-[hash].html) that was not referenced anywhere in the site's public HTML source.
Why is this risky?	S3 buckets frequently contain a mix of public web assets and private backend files. Allowing unauthenticated users to map the entire structure of the bucket circumvents intended application routing and exposes "security by obscurity" files to immediate discovery.
Potential Impact	If this misconfiguration existed in a production environment, unauthorized users could effortlessly download sensitive database backups, proprietary source code, or internal credentials accidentally stored in the bucket.
Recommendation	Audit the S3 bucket's IAM policy and Access Control Lists (ACLs). Revoke the s3:ListBucket permission for the Everyone (or *) principal. Ensure AWS "Block Public Access" settings are enabled at the account or bucket level unless explicitly required for a specific, isolated use case.
Reference / Source	flaws.cloud (Level 1); AWS S3 Security Best Practices.

Finding CLOUD-03: Publicly Exposed S3 Bucket via Wildcard Principal Policy

Finding ID	CLOUD-03
Title	Publicly Exposed S3 Bucket via Wildcard Principal Policy
Cloud Service	AWS IAM / S3
Severity	High
Description	Review of the resource-based bucket policy for thebigiamchallenge-storage-9979f4b revealed a critical trust boundary misconfiguration. The policy grants both s3:GetObject and s3:ListBucket to "Principal": "*". By specifying the exact prefix files/ defined in the policy's condition, an unauthenticated AWS CLI request (--no-sign-request) successfully listed the directory and downloaded flag1.txt directly to standard output.
Why is this risky?	Using "Principal": "*" in an S3 bucket policy without restricting access via IP address or a VPC endpoint makes the data accessible to the entire internet.
Potential Impact	An external attacker can enumerate allowed prefixes and stream or download all objects within them, leading to a complete breach of data confidentiality.
Recommendation	Remove the "Principal": "*" statements. If the bucket is meant to be private, explicitly define trusted AWS Account IDs. If it is meant to host public assets, enforce Block Public Access at the account level and use CloudFront with Origin Access Control (OAC) to serve the files securely.
Reference / Source	The Big IAM Challenge; AWS Well-Architected Framework (Security Pillar).

6. Risk

Finding ID	Vulnerability	Severity	Likelihood	Justification
CLOUD-01	Overly permissive IAM policy (wildcards)	Medium	Medium	Requires the identity to already be compromised, but then grants very broad follow-on access.
CLOUD-02	Publicly listable S3 bucket (flaws.cloud)	High	High	No authentication required at all; any internet user can enumerate and read hidden files.
CLOUD-03	Public S3 bucket via wildcard Principal	High	High	No authentication required; unauthenticated CLI access directly yields file contents.

7. Impact

Taken together, the three findings illustrate how cloud misconfigurations compound across the identity and storage layers. An overly permissive IAM policy widens the blast radius of any single compromised credential, while publicly exposed S3 buckets - whether through bucket ACLs or wildcard Principal statements in bucket policies - remove the need for any credential compromise at all. In a real production environment, this combination would let an attacker move from simple reconnaissance (listing a bucket) to full data exfiltration within minutes, and would let a compromised low-privilege identity escalate its effective reach across an AWS Organization.

8. Recommendation

- Apply the principle of least privilege to all IAM policies: replace wildcard actions and resources with explicit, scoped API actions and specific resource ARNs.
- Enable AWS “Block Public Access” at the account level so that no bucket policy or ACL can accidentally expose data publicly.
- Remove any "Principal": "*" statements from S3 bucket policies; scope access to specific, trusted AWS account IDs or IAM roles.
- Serve static website content through a CDN (e.g., CloudFront) using Origin Access Control (OAC) rather than making the origin bucket itself publicly readable.
- Regularly audit IAM policy versions and S3 bucket policies as part of routine cloud security hygiene, using tools such as AWS IAM Access Analyzer.

9. Evidence

All screenshots referenced in this report (Figures 1–6) were captured directly from my own lab sessions on Pwned Labs, flaws.cloud, and The Big IAM Challenge, and are embedded in Section 4 (Lab Summaries) alongside the finding they support. The system clock visible in Figure 6 (19-Sep-26) satisfies the academic-integrity requirement to show the current date/time in at least one screenshot per completed lab.

10. Cloud Security Questions

1. What is the Shared Responsibility Model? Give one responsibility of the cloud provider and one responsibility of the customer.

The Shared Responsibility Model dictates that cloud security is a joint effort. The cloud provider is responsible for security of the cloud (e.g., physical data center security, hardware maintenance). The customer is responsible for security in the cloud (e.g., configuring IAM policies, securing customer data, and patching guest operating systems).

2. What is IAM and why is least privilege important?

Identity and Access Management (IAM) is the service that controls who is authenticated and authorized to access cloud resources. The principle of least privilege is critical because it ensures identities are only granted the absolute minimum permissions necessary to perform their required tasks, drastically reducing the “blast radius” if an account is compromised.

3. What is the security difference between a private and publicly accessible storage resource?

A private storage resource inherently blocks all access unless a user or service is explicitly authenticated and granted permission via IAM or resource policies. A publicly accessible storage resource allows anonymous, unauthenticated internet users to read (and sometimes write) data, creating a severe risk of data exfiltration if sensitive information is stored there.

4. What is an IAM policy? Name the main elements of a policy statement.

An IAM policy is a JSON document that explicitly defines permissions within the cloud environment. The main elements of a policy statement are Effect (Allow or Deny), Action (the specific API calls permitted, like s3:GetObject), Resource (the specific Amazon Resource Names the actions apply to), and optionally the Condition (when the policy applies).

5. What is one cloud-security control you would recommend for protecting sensitive storage?

Enable the “Block Public Access” feature at the account or bucket level. This acts as a master override to prevent any bucket policies or Access Control Lists (ACLs) from accidentally making the storage publicly readable.

6. Why should cloud credentials and access keys be protected and monitored?

Cloud credentials (like AWS Access Keys) provide programmatic, remote access to the cloud environment. If leaked (e.g., in a public GitHub repository), attackers can use them to exfiltrate sensitive databases, delete infrastructure, or spin up expensive compute instances for cryptomining, leading to massive financial and reputational damage.

11. Conclusion

This assignment reinforced that cloud security failures rarely stem from exotic attack techniques - they stem from permissions and access controls that are broader than necessary. Across all three labs, the recurring theme was the same: wildcards in IAM actions/resources and wildcards in S3 bucket Principals both remove the boundaries that least-privilege access is supposed to enforce, whether on the identity side or the storage side. Practicing enumeration with the AWS CLI (`aws sts get-caller-identity`, `aws iam get-user`, `aws iam get-policy-version`, `aws s3 ls --no-sign-request`) made it concrete how quickly an attacker can map an environment once any single weak point - a leaked credential or an over-permissive bucket policy - is found.

What I would improve next: practice using AWS IAM Access Analyzer and automated tools (e.g., Prowler or ScoutSuite) to detect these same misconfigurations at scale rather than manually reviewing individual policy documents, and extend this practice to Azure and GCP equivalents of IAM and object storage to build a more complete cross-cloud security picture.